3D catoms meta-modules locomotion

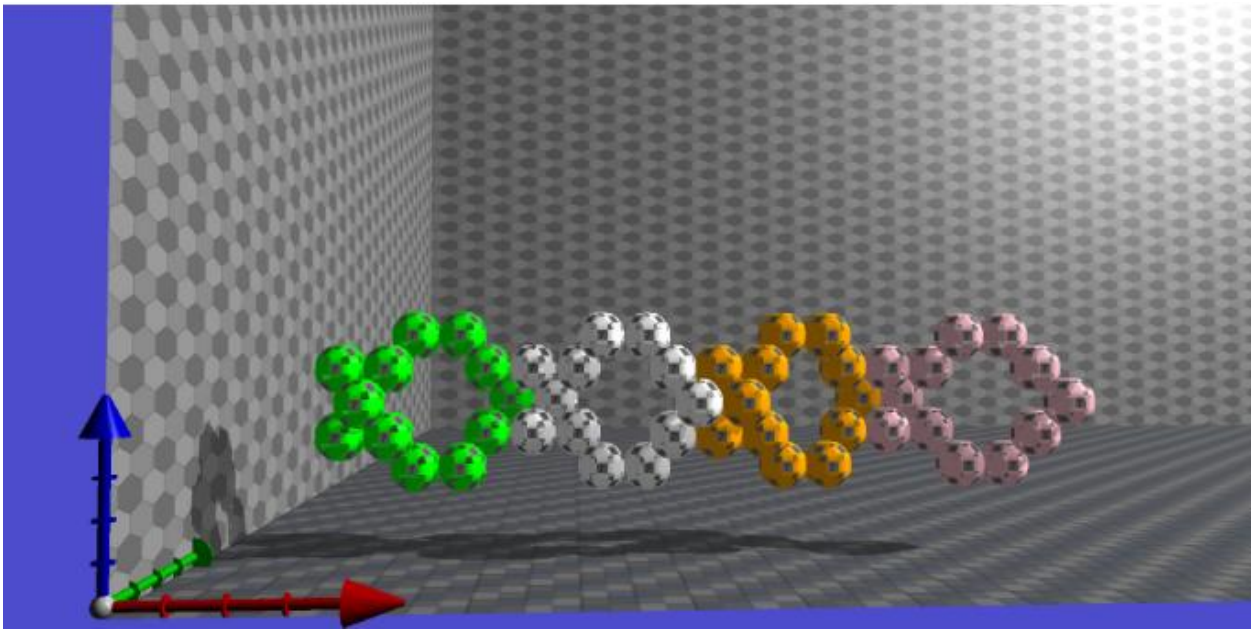
Work to return:

1. The source code for your implementation.
2. A demonstration video showcasing the execution of your algorithm.
3. A detailed report written in LaTeX, explaining your algorithm along with an evaluation of its time complexity, message complexity, and the number of motions. Ensure to include the names of all contributors in the report.

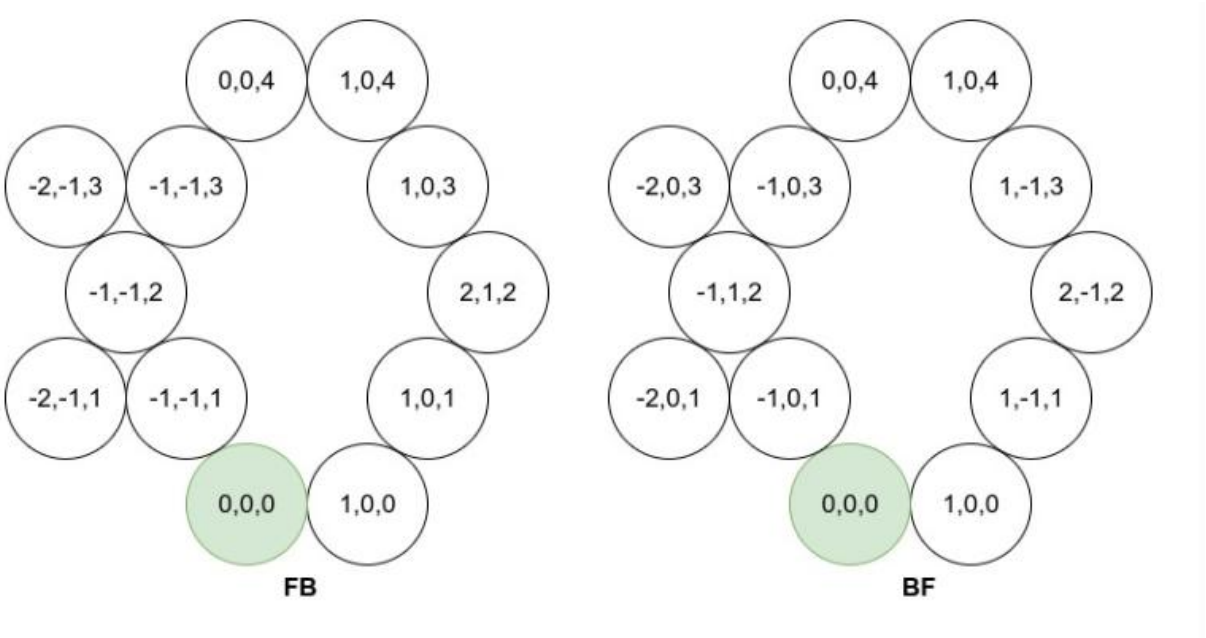
Objective

Design a mechanism to enable a set of 3D catoms, arranged in a meta-module structure, to move in a specified direction.

- Starting with a linear configuration, dismantle the left-most meta-module (highlighted in green), transport its individual modules through the structure, and reassemble it on the right side (adjacent to the pink module).



- The meta-modules can be in two symetrical shapes FrontBack (FB) and BackFront (BF).



Helpful Code:

- Use the following line to check the neighborhood state of a module. This can help determine whether the module should be dismantled or if a meta-module needs to be assembled on its right.

```
module->getLocalNeighborhoodState().to_ulong();
```

- The following are the relative coordinates for each meta-module shape (FB and BF). These coordinates are arranged in a specific order to facilitate seamless disassembly and assembly processes, ensuring no blockages occur during the transitions.

```
static const std::array<Cell3DPosition, 12> FB_RELATIVE_POSITIONS = {
   Cell3DPosition(-2, -1, 3), Cell3DPosition(-1, -1, 2), Cell3DPosition(-2, -1, 1),
   Cell3DPosition(-1, -1, 3), Cell3DPosition(-1, -1, 1), Cell3DPosition(0, 0, 4),
   Cell3DPosition(0, 0, 0),   Cell3DPosition(1, 0, 4),   Cell3DPosition(1, 0, 0),
   Cell3DPosition(1, 0, 3),   Cell3DPosition(1, 0, 1),   Cell3DPosition(2, 1, 2)};


static const std::array<Cell3DPosition, 12> BF_RELATIVE_POSITIONS = {
   Cell3DPosition(-2, 0, 3), Cell3DPosition(-1, 1, 2), Cell3DPosition(-2, 0, 1),
   Cell3DPosition(-1, 0, 3), Cell3DPosition(-1, 0, 1), Cell3DPosition(0, 0, 4),
```

Cell3DPosition(0, 0, 0), Cell3DPosition(1, 0, 4), Cell3DPosition(1, 0, 0),

Cell3DPosition(1, -1, 3), Cell3DPosition(1, -1, 1), Cell3DPosition(2, -1, 2)};

- To perform a 3D catom rotation:

getScheduler()->schedule(new Catoms3DRotationStartEvent(getScheduler()->now() + 100, module, nextPosition);

- You can use BFS to find a path between a start and target positions (You can also try more efficient algorithms like A*)

```
vector<Cell3DPosition> MMLocomotionBlockCode::findPath(Cell3DPosition &start,
                                                       Cell3DPosition &goal) {
  // BFS that return the path from start to goal and use getAllPossibleMotionsFromPosition
  vector<Cell3DPosition> path;
  set<Cell3DPosition> visited;
  map<Cell3DPosition, Cell3DPosition> parentMap;
  queue<Cell3DPosition> q;
  q.push(start);
  visited.insert(start);
  bool found = false;
  while(!q.empty() and !found) {
    Cell3DPosition current = q.front();
    q.pop();
    if(current == goal) {
      found = true;
      break;
    }
    vector<Cell3DPosition> reachablePositions;
```

```
      bool success = getAllPossibleMotionsFromPosition(current, reachablePositions);
      if(success) {
        for(auto &pos: reachablePositions) {
          if(visited.find(pos) == visited.end()) {
            visited.insert(pos);
            parentMap[pos] = current;
            q.push(pos);
          }
        }
      }
    }
    if(found) {
      cout << "Found path from " << start << " to " << goal << "\n";
      Cell3DPosition step = goal;
      while(step != start) {
        path.push_back(step);
        step = parentMap[step];
      }
      reverse(path.begin(), path.end());
    } else {
      cout << "No path found from " << start << " to " << goal << "\n";
    }
    return path;
}

bool MMLocomotionBlockCode::getAllPossibleMotionsFromPosition(
```

```
  // get all reachable positions from any given position (used in findPath)
  Cell3DPosition position,
  vector<Cell3DPosition> &reachablePositions) {

  Catoms3DBlock* mod = static_cast<Catoms3DBlock*>(lattice->getBlock(position));
  if(mod) {
    for(auto &neighPos: lattice->getFreeNeighborCells(position)) {
      if(mod->canMoveTo(neighPos)) {
        reachablePositions.push_back(neighPos);
      }
    }
    return !reachablePositions.empty();
  }
  bool found = false;
  for(auto &neighPos: lattice->getActiveNeighborCells(position)) {
    Catoms3DBlock* neigh = static_cast<Catoms3DBlock*>(lattice->getBlock(neighPos));
    vector<Catoms3DMotionRulesLink*> vec;
    Catoms3DMotionRules motionRulesInstance;
    Cell3DPosition pos;
    short conFrom = neigh->getConnectorId(position);
    motionRulesInstance.getValidMotionListFromPivot(neigh, conFrom, vec,
static_cast<FCCLattice*>(lattice), NULL);
    for(auto link: vec) {
      Cell3DPosition toPos;
      neigh->getNeighborPos(link->getConToID(), toPos);
      reachablePositions.push_back(toPos);
```

```
            found = true;
        }
    }
    return found;
}
```